

Presentation Pack — Assignment Answers + 5-Minute Script

*Two documents in one. **Part A** answers every question the brief asks, in depth, with concrete code/data references — read this before the interview. **Part B** is a continuous 5-minute script you can speak end-to-end during the demo.*

PART A — Detailed answers to every question in the brief

A1. Who are the users, and what is the job-to-be-done for daily vs weekly?

The user. A D2C brand's growth lead or founder. Reads on a phone, between 8 and 9 AM, before their first standup. Not doing analysis — doing **triage**. They have 90 seconds to decide whether to act on anything before the team syncs.

The Daily Digest job: answer "do I need to act today?". The digest is opinionated — it leads with the single most anomalous signal from yesterday, flags whether the data is complete, and gives a directional interpretation with hedging proportional to the evidence. A founder doing INR 22.6 lakh median daily revenue does not need to know impressions were up 3%. They need to know that new-customer AOV on Dec 10 jumped 190.5% week-over-week, and that this pattern is consistent with a sale event drawing in a discounted-acquisition cohort rather than organic demand. After 90 seconds of reading they walk into standup with a specific question rather than a vague feeling that the numbers looked different.

The Weekly Report job: answer "what is this week's story?". Different reader mode — CMO, agency lead, or founder doing Sunday planning. Not triage; pattern recognition across seven days. Which channels showed sustained movement? Was a single anomalous day noise, or the start of a trend? Should budget allocation be revisited before next week?

The critical distinction is temporal compression. Daily answers "was yesterday unusual?". Weekly answers "what is the underlying story?". The same number means different things. A campaign showing ROAS = 0.000 for the week of Feb 1 (search_google_002 spent budget and returned zero attributed orders) belongs in the Weekly Report with full context — it is a budget decision, not a morning alert. A weekly report tolerates more text, ranked tables, and explicit "sustained" vs "one-day spike" labelling. A daily digest does not.

A2. How do we decide which metric movements are "important"?

The scoring formula

$$\text{final_score} = (|z|/3 \times 0.4 + |\text{WoW}|/100 \times 0.4 + |\text{MoM}|/100 \times 0.2) \times \text{business_weight}$$

Three statistical signals weighted by their actionability, then multiplied by a business-importance weight.

What each signal catches

Z-score (40% weight). How far today's value is from the recent baseline, in units of the metric's own standard deviation. $Z = 2$ means today is in the top 2.5% of recent days for this metric. **Key implementation detail:** the z-score uses *same-day-of-week* mean AND *same-day-of-week* standard deviation. Both the numerator and denominator come from the same distribution — using the overall 28-day std with a same-DoW mean was a bug I caught and fixed in code review (it systematically understates z on stable metrics). On this dataset DoW adjustment is mandatory because Thursdays are the strongest day and Sundays the weakest, with a ~40% spread. Unadjusted z would flag every Sunday as a downside anomaly.

Week-over-week delta (40% weight). Catches sharp seven-day reversals that z-score might smooth over. Especially important for metrics with strong weekly seasonality.

Month-over-month delta (20% weight). Catches slow drifts. A metric that climbs 2% per week for six weeks has a low z-score (the rolling mean climbs with it) but a clear MoM trend.

The 40/40/20 split reflects that D2C operational decisions are made on a 7-day horizon, not a 30-day one.

Business weights

Metric tier	Weights	Why
Top tier	revenue 1.0, orders 0.9	Direct P&L; the only metrics that map to outcomes
Decision tier	ROAS 0.8, AOV 0.75, spend 0.7	Drive same-day budget/pricing decisions
Health tier	new_customer_share 0.65, attributed_revenue 0.6	Acquisition and retention signals
Supporting tier	CPC 0.4, CPM 0.35, CTR 0.3	Auction and creative signals; rarely actionable same-day
Scale tier	impressions 0.2, clicks 0.2	High noise, low decision value

This asymmetry is the answer to "statistical significance $\neq$ business significance" from the brief. A 50% impressions swing is statistically anomalous but is usually auction dynamics, day-of-week, or pacing — and almost never needs a same-day decision. A 5% revenue dip is barely statistically anomalous and very actionable.

What guards against scoring artifacts

Dedup table (`DEDUP_PAIRS`). Shopify emits the same revenue number as three overlapping slices — daily total, channel slice (~91% goes into `unattributed`), customer-type slice. Scoring all three would triple-count the same revenue and let the largest single row dominate every digest. The dedup table keeps the daily total as the headline and drops the `unattributed` channel slice; attributed slices remain as directional signals.

Ratio cap at $\pm 500\%$. ROAS, CTR, CPC, CPM explode mathematically when the denominator approaches zero. `shopping_google_003` in the raw data shows WoW ROAS = +4,653% because its

prior-week spend was INR 340. That number is a near-zero-denominator artifact, not a 47× efficiency improvement. The cap eliminates the artifact tail; capped values are flagged `[capped]` in the fact sentence so the LLM communicates the caveat instead of treating it as a business insight.

Last-day data-quality flag. Any finding whose target date equals the dataset's last date is marked `is_data_quality_flag = True`, sorted to the bottom of the ranking, and annotated with a caveat note. The Feb 7 in this dataset shows revenue "down 92%" — almost certainly because the export was generated mid-day. Without this flag, the system would lead with a false alarm.

What signals feed the ranker (the brief's exact wording)

- **Statistical:** z-score with DoW adjustment, WoW delta, MoM delta.
- **Business-rule:** the weights table above, the dedup table, the ratio cap, the data-quality flag.
- **Behavioral:** not yet implemented. The architecture accepts per-customer weight overrides; once engagement data exists (clicks, follow-ups, "not relevant" flags), per-customer learned weights replace the global defaults.

A3. How does personalization work — cold start → warm → mature?

Cold start (under 14 days of data)

Fixed defaults. Business weights from the global table. DoW adjustment is disabled because there aren't enough same-weekday observations to compute a stable baseline. Z-scores are replaced with percentile rank within the available window. The output leads with the disclosure "baseline statistics stabilise after 4 weeks of data" so the user calibrates their expectations.

Profile defaults to **PROFILE_GROWTH** on the assumption that a customer onboarding to an analytics tool is more likely to be in acquisition mode than retention mode. The user can override at signup with one form field: brand stage.

Warm (2 to 8 weeks)

At 2 weeks DoW adjustment activates — at least two observations per weekday. At 4 weeks the full 28-day rolling window fills and z-scores become statistically valid. The interesting work during this window is collecting **profile signal**:

- Which findings did the user click into / expand?
- Which findings did they scroll past?
- Which findings triggered a follow-up question?
- Which metrics were they viewing in the dashboard the same day?

After 4–6 weeks of these implicit signals, fit a simple per-customer logistic regression on "did the user engage with this finding?". The coefficients become per-customer business weights, replacing the global defaults.

None of this is implemented in the current prototype — no engagement data exists yet. The ranker architecture is ready to accept per-customer weight overrides when the data does.

Mature (60+ days)

Three things become possible only at this horizon:

1. **Seasonality detection.** This specific brand's recurring patterns — every Thursday is strong, every Sunday is weak, the 15th of each month is a payroll spike — stop being flagged as anomalies. The system learns the brand's normal.
2. **Campaign persistence classification.** Ephemeral test campaigns (run for a week, then turned off) are filtered out of the ranker entirely. Persistent campaigns are scored against their own history.
3. **Engagement-based weight learning.** Per-customer business weights from 60+ days of click data. A scale-stage brand that consistently engages with retention findings gets retention-tilted weights even though its global profile is "scale".

A mature digest is shorter than a cold-start digest — the system has learned what to suppress.

What's actually built today

Two profile re-rankers and the prompt-injection layer. Personalisation operates in two places.

Re-ranking at the online layer. `apply_profile_reranking()` multiplies `final_score` by 1.3 for any finding whose metric is in the customer's `primary_metrics`, then re-sorts. The same Dec 10 dataset produces a different ranked order for growth vs scale.

Prompt injection. The profile's priorities and known concerns are interpolated into the user message so the LLM weights narrative emphasis the same direction the re-ranker did.

The 1.3× multiplier is the cold-start prior — auditable, easy to explain, ready to be replaced by a learned per-customer weight once engagement data exists. **It is not a learned model and I'm explicit about that.**

A4. System architecture — batch vs online, deterministic vs generative

Two tiers



Why this split. Batch operations are predictable, parallelizable, and cheap. They should do as much work as possible. The LLM call is the expensive, slow, non-deterministic stage. It should do as little work as possible — just the explanation. Putting the ranking inside the LLM call would make every report generation a 30-second operation with non-deterministic output and 10× the cost.

The deterministic / generative boundary

The boundary sits at `ranked_findings.json`. Everything upstream is deterministic and auditable — every number that appears in the final report was computed before the LLM was invoked. The LLM's job is exclusively to write hedged prose that connects pre-computed facts.

Operation	Python	LLM
Loading data	✓	
Computing z-scores, WoW, MoM	✓	
Applying business weights	✓	
Sorting and selecting top N	✓	
Formatting fact sentences	✓	
Numeric grounding validation	✓	
Writing the headline		✓
Connecting two findings into a story		✓
Phrasing the investigation question		✓
Choosing tone for the customer profile		✓

The reason for the boundary is unspectacular. LLMs are unreliable calculators and reliable writers. Asking an LLM to decide whether a ROAS movement is significant requires it to reason about baselines, volatility, and business context — tasks it gets wrong in unpredictable ways. Asking it to write "search_google_002 recorded a ROAS of 0.000 for the week of February 1, meaning spend occurred with zero attributed conversions" is a task it performs consistently and well.

Cost mechanics at scale

Prompt caching — the system prompt is identical across every report. `cache_control: ephemeral` caches it server-side and bills cache reads at ~10% of input cost within ~5 minutes. For batch report generation this is roughly an 80% reduction on input-token cost.

Tiered model strategy — Haiku for low-engagement customers (~90% of base on quality the task can tolerate), Sonnet for high-value accounts. Drops total cost roughly 60% further.

At 10,000 customers with two reports each per day: without caching, ~₹15 lakhs/month on LLM spend. With caching + tiered model, ~₹4 lakhs/month.

A5. Top 3 failure modes and mitigations

Failure 1 — Hallucinated causation

The risk. The LLM writes "revenue dropped because Meta CPM rose 18%." The number is correct, the framing is fabricated. The brand owner cuts Meta. Revenue keeps dropping because the real cause was a broken checkout. Trust gone.

Why it would happen by default. LLMs are trained on millions of analytical reports where "X dropped because Y rose" is overwhelmingly common. Without instruction the model defaults to confident causation.

Three-layer mitigation.

1. **System prompt rules.** "Because", "caused by", "due to" are banned. Allowed hedge phrases listed explicitly: "may indicate", "worth investigating whether", "one possible driver".
2. **Cited constraint.** The prompt states the actual evidence — $r = 0.12$ between spend and revenue in this data — so the model understands *why* the rule exists.
3. **Numeric grounding validator.** After generation, `validate_numeric_grounding()` extracts every numeric token from the LLM output, normalises (strip commas, drop %, round to 2dp, skip 1900–2100 years), and verifies each appears in the FACTS, STEADY, user-prompt, or system-prompt context. Ungrounded numbers reject the output and trigger `render_template_fallback()`. The check is logged with the specific tokens, so the failure mode is debuggable.

Evidence it works. On Dec 10's report, the model wrote: "Total Shopify revenue is INR 3,915,420 (+167.6% WoW). Ad spend and Shopify revenue are weakly correlated ($r \sim 0.12$) — the two moved in the same direction today, but this does not confirm that paid acquisition drove the result." Zero uses of "because". The hedging is the desired output.

Failure 2 — Alert fatigue

The risk. A brand with 15 active campaigns across two ad platforms receives a digest with 40+ findings every morning. Users stop reading within a week.

Why it would happen. Naive surfacing of every $|z| > 2$ metric on this dataset flags 5–10 metrics every day before any seasonality or weighting is applied.

Five compounding mitigations.

1. **DoW adjustment** prevents Sunday-naturally-low from flagging every Sunday. Most of the false-positive volume disappears here.
2. **Business weights** ensure impressions $z=2.5$ can't beat revenue $z=2.0$.
3. **Top-N cap** (5 daily, 8 weekly) is a hard limit regardless of how many signals exceed threshold.
4. **Sustained-signal boost** in the weekly ranker promotes 3-day trends ahead of one-off spikes, so the weekly tells the underlying story rather than the loudest one.
5. **Mandatory "what's holding steady" section**, grounded in a separate STEADY FACTS block, gives the reader explicit reassurance that not everything is on fire.

Failure 3 — Incomplete last-day data

The risk. Feb 7 (the dataset's last date) shows Shopify revenue down 92%. That looks catastrophic. It is almost certainly because the export was generated mid-day and the day wasn't complete. If the system leads with "revenue down 92%", the brand owner panics, opens an incident, escalates. Six hours later somebody figures out the export was mid-day.

Three-layer mitigation.

1. `is_data_quality_flag = True` is set on findings whose target date equals the dataset's last date.
2. **Sort to bottom.** Flagged findings appear at the bottom of the ranked list — they don't disappear (the user might still want to see them) but they don't lead.

3. **Fact-sentence caveat.** The fact sentence is annotated

[NOTE: possible incomplete data – last day of dataset] so the LLM communicates the caveat explicitly rather than presenting it as a confirmed movement.

In production this last-day heuristic would be replaced by a proper freshness check (export timestamp, expected row count, deviation from prior same-DoW). For the prototype the heuristic is a reasonable proxy.

A6. What would I measure to know this is working?

Product metrics, not model metrics. The model isn't the product — the morning habit is.

Open rate. Does the customer open the digest every morning, or only when something is wrong? An open rate that decays over a fortnight is the leading indicator of alert fatigue or irrelevance, regardless of what the model thinks of its own output.

Follow-up rate. Did a finding spark an investigation — a chat-thread question, a metric drill-down, a teammate Slack? Closest proxy for "the system surfaced something useful" that doesn't require explicit user feedback.

Not-relevant rate. When a thumbs-down or "hide this" affordance exists, what fraction of findings get dismissed? Per finding type and per customer segment, this is the input signal for replacing the business-weight defaults with learned per-customer weights.

7-day retention after first digest. Does the digest change the morning habit, or is it tried-and-abandoned? This is the single most important number; everything else is a leading indicator of this one.

Alert-to-action conversion. When the system flags something with an "investigate today" question, did the user take an action that's traceable in the data (paused a campaign, changed a budget, opened a checkout audit)? Hard to measure without integration; it's the metric a CFO would actually care about.

Why I would not measure model metrics. BLEU, ROUGE, or judge-LLM evaluations correlate poorly with whether the brand owner actually uses the product. A digest that scores 0.92 on lexical overlap and gets ignored every morning is a failed product. A digest that reads slightly awkwardly but produces six chat questions per week is a successful product. Optimising the wrong number is the failure mode here.

A7. How I navigated each "key tension" from the brief

Signal vs noise

A 20% revenue dip on a Tuesday might be normal seasonality; a 3% dip might be a checkout bug. The system handles this via **z-score normalised by each metric's own volatility**. A 3% drop on a stable metric scores higher than a 20% drop on a noisy one. The DoW adjustment then strips out seasonality before the z is computed.

Personalization without cold-start failure

Two profiles at signup based on brand stage (one form field). The profile is a reasonable prior — most growth-stage brands really do care about acquisition. Better than no personalisation; worse than learned personalisation. **Buys time** while engagement data accumulates for the warm-customer learning stage.

Explanation vs speculation

The system errs hard on the side of epistemic honesty. **Causal language banned**, hedge phrases mandated, numeric grounding validator catches invented numbers, the $r=0.12$ constraint cited in the prompt so the model understands *why* the rule exists. Result: every causal claim in the output is hedged. Some hedging is a price — the digest occasionally feels less confident than a real analyst would. That price is correct because confident causation when the data doesn't support it is the failure mode that kills trust.

Daily ≠ weekly compressed

Different ranker output for daily and weekly. Daily uses `rank_day` and surfaces the top 5 most-anomalous-yesterday findings. Weekly uses `rank_week` which aggregates each metric's daily scores across seven days and **boosts metrics that appeared in the top for 3+ days** (sustained-signal boost). A sharp one-day spike that disappears the next day leads the daily but does not lead the weekly. A small steady drift over five days leads the weekly but doesn't lead any single daily.

A8. The v1 → v2 architectural evolution

When asked "is your ranking approach the best, or what would you do differently?", I lead with three specific, named, industry-validated upgrades — not vague more-ML.

Upgrade 1 — STL or Prophet for the baseline

What v1 does: 28-day rolling mean with same-day-of-week adjustment. Correct for the prototype scope.

What v1 can't do: handle annual seasonality (Diwali, EOSS, Christmas), adapt fast to genuine level shifts (new product launch doubling revenue), or stay robust when one bad day is in the baseline window.

v2 replacement: STL decomposition (Cleveland et al. 1990) or Prophet (Meta's open-source extension with explicit holiday calendars and changepoint detection). The pipeline:

```
For each metric series, nightly:
trend, seasonal, residual = STL(series, period=7).fit()
z_score = (today_value - trend(today) - seasonal(today)) / std(residual)
# downstream stays identical
```

The interface contract with the LLM doesn't change — only the upstream z computation is more honest.

Production usage: Netflix (streaming-quality anomaly detection), Uber (Prophet for capacity), LinkedIn (Luminol library), DataDog Watchdog, Booking.com engineering blog.

Why not in v1: STL needs at least two cycles of the longest seasonality. Annual seasonality needs 2+ years of data; the current dataset is 160 days.

Upgrade 2 — CausalImpact for the attribution question

What v1 does: hedges causation. "Revenue and Meta spend moved together but this does not confirm paid acquisition drove the result." Correct, but unsatisfying.

What v1 can't do: answer "did my budget change drive revenue?" with a number. The brand owner gets a disclaimer.

v2 addition: CausalImpact — Google's Bayesian Structural Time Series library (Brodersen et al. 2015). Pipeline:

```
For each flagged or detected intervention:
  pre = series before intervention_date
  ctrl = a series that wasn't affected by the intervention
  model = CausalImpact(target=series, control=ctrl, intervention=intervention_date).fit()
  # surface: "Meta budget +50% on Dec 4 → revenue +12% over counterfactual
  #          (95% CI: 7-17%, posterior P(lift>0) = 0.97)"
```

This converts hedging into decision-grade attribution.

Production usage: Google (introduced the technique, uses internally), Walmart Global Tech (published case study), Lifesight, Measured, Recast (all major incrementality vendors), Meta's Robyn (related state-space modelling).

Why not in v1: the dataset has no labelled interventions and the channels overlap in customer base, so there's no clean control series.

Upgrade 3 — Cross-metric story grouping

What v1 does: independent findings. Five findings = five bullets.

What v1 can't do: recognise that Meta CPM +18%, Meta CPC +14%, Meta CTR -8% and Meta ROAS -22% on the same day are **one story** (auction pressure), not four findings.

v2 addition: clustering step between ranker and LLM. Pipeline:

```
After top-N + profile rerank:
  similarity[i,j] = 0.4·same_source + 0.3·same_channel + 0.2·same_campaign + 0.1·matched_direction
  clusters = connected_components(similarity > 0.7)
  for each cluster of 2+ findings:
    story_finding = synthesize_story_headline(cluster) # tiny LLM call or rule-based template
```

The user-facing digest collapses four bullets into one story: *"Auction pressure on retargeting_meta_001 today — CPM, CPC, and CTR all moved against you, ROAS dropping 22%."*

Production usage: Anodot (their core "stories" feature), Outlier.ai (acquired by Salesforce), OutOfTheBlue's incident summary card, DataDog Watchdog correlated-alert grouping, Splunk ITSI.

Why not in v1: at 5-10 findings per day, the marginal value of grouping is small; the design lands the win at >20 findings per day or multi-tenant scale.

What stays the same in v2

The architectural skeleton is method-agnostic: - Python computes, LLM explains — independent of scoring method - Numeric grounding validator — independent of scoring method - Retry, fallback, prompt caching — all independent - 3-level sort key, top-N slicing, profile rerank — compose with any upstream signal

The upgrades change **how the z-score is computed** and **what additional findings get surfaced**. They don't change the architecture.

Full v1-to-v2 narrative with research path and enterprise comparison table is in `OPTIMIZATION_JOURNEY.md`.

PART B — The 5-minute presentation script

*Approximately 750 words at conversational pace (~150 wpm). Each section is a beat — pause briefly between beats. Numbers in **bold** are the ones you point at on screen.*

Beat 1 — Open the frame (20 seconds)

"Thank you. I'll walk you through the system in five minutes, then we can go as deep as you want on any part. The one-line version: it's a daily and weekly performance digest for D2C brands, with a hard architectural split — **Python computes, the LLM only explains**. Every number in the output is pre-computed and validated. That single design choice is what separates this from a generic LLM-on-a-dashboard prototype. Let me show you why it matters."

Beat 2 — The problem (35 seconds)

"A typical D2C brand produces around **200 metric values per day** — revenue by channel, ROAS by campaign, CPM, CTR, AOV, new-customer share, and so on. The brand owner opens their phone at 8 AM and has **90 seconds** before standup. They are not doing analysis. They are doing triage. The question they need answered is 'do I need to act on anything before my team syncs today?'. Dashboards make them figure that out themselves. My system answers it for them — surfaces the **3 to 5 things** that actually mattered yesterday, explains them in plain English with hedging proportional to the evidence, and ends with one specific question to investigate today."

Beat 3 — Architecture (50 seconds)

"There are two stages. The first is a deterministic Python ranker. It scores every metric movement using three signals — **z-score against a same-day-of-week baseline**, week-over-week delta, month-over-month delta — and multiplies by a business-importance weight. Returns the top 5 findings, formatted as fact sentences with all the numbers already in them.

The second stage is a Claude API call. It takes those fact sentences, attaches the customer profile, and writes the digest in a strict format. The LLM never picks the findings. It never computes a percentage. It only writes prose connecting pre-computed facts.

This separation does three things. **One** — hallucinated numbers become impossible because the LLM never has to compute. **Two** — every ranking decision is auditable end-to-end in the logs. **Three** — the expensive stage does the minimum work, so cost and latency stay manageable at scale."

Beat 4 — Why the design choices are what they are (45 seconds)

"Three things shaped every decision. First, the **correlation between Meta ad spend and Shopify revenue in this data is 0.12** — effectively no relationship. That single number is why my prompt bans the words 'because' and 'caused by' — causation cannot be inferred at r equals 0.12, so writing it would be confidently wrong.

Second, **91% of Shopify orders are unattributed** at the channel level. Channel revenue is a directional signal at best — not a clean attribution model. That forced a deduplication rule, which I'll come back to.

Third, **Thursdays are the strongest day of the week and Sundays the weakest, with a 40% spread**. Without day-of-week-adjusted baselines, every Sunday would flag as a downside anomaly. The DoW adjustment compares Sunday to recent Sundays, not to a mixed mean."

Beat 5 — Personalisation that's real, not decorative (35 seconds)

"Personalisation operates in two places. At the online layer, `apply_profile_reranking()` multiplies the score by 1.3 for any finding whose metric is in the customer's priority list. Growth profile leads with ROAS, new-customer share, attributed revenue. Scale profile leads with AOV, returning orders, retention metrics. **Same data, different leading findings** for the two profiles. This is the cold-start prior — auditable, easy to explain, ready to be replaced by per-customer learned weights once engagement data exists. **It is not a learned model and I'm explicit about that.**"

Beat 6 — Hallucination prevention (35 seconds)

"Three layers. **The prompt bans causal words and lists allowed hedge phrases**. It cites the $r=0.12$ constraint so the model understands why. After generation, a numeric grounding validator extracts every number from the output and verifies it appears in the FACTS block, the STEADY block, the user prompt metadata, or the system prompt. If any number is ungrounded, the output is rejected and a deterministic template fallback fires. **The promise of zero hallucination becomes verifiable**, not just a wish."

Beat 7 — Three failure modes I designed against (50 seconds)

"Hallucinated causation — mitigated by the three layers I just described. **Alert fatigue** — a brand with 15 campaigns would get 40 findings a day without DoW adjustment, business weights, a top-N cap, sustained-signal boosting for the weekly, and a mandatory 'what's holding steady' section. **Incomplete last-day data** — the Feb 7 in this dataset shows revenue down 92% because the export was mid-day. A naive system would lead with that. My system flags the last date as data-quality-suspect, sorts those findings to the bottom, and appends an explicit caveat to the fact sentence."

Beat 8 — Where this breaks at scale (35 seconds)

"At 100 customers it's fine. At **10,000 customers** the binding constraint is cost — about USD 110k per month on Sonnet without caching. **Prompt caching on the shared system prompt cuts that by roughly 80%**. A tiered model strategy — Haiku for low-engagement customers, Sonnet for high-value accounts — cuts further. With both, total LLM spend at 10,000 customers drops from 15 lakhs per month to about 4 lakhs."

Beat 9 — The v2 architecture I'd ship next (35 seconds)

"Three specific upgrades, not vague more-ML. **First — replace the rolling baseline with STL or Prophet decomposition**. Trend, seasonality, and residual handled properly; holidays modelled explicitly. Netflix, Uber, DataDog Watchdog all use this pattern. **Second — add CausalImpact for the attribution question**. Bayesian structural time series, from Google's 2015 paper. Instead of hedging at

$r=0.12$ I'd give a counterfactual estimate with credible intervals — Walmart, Lifesight, every serious incrementality vendor uses this. **Third — cross-metric story grouping.** Four correlated findings should become one story bullet headlined by the common cause. Anodot and OutOfTheBlue both do this. None of these are speculative — they're the standard production upgrades. I didn't ship them in v1 because the dataset doesn't support them yet: STL needs two cycles of annual seasonality, CausalImpact needs labelled interventions, and story grouping has marginal value at 160-day single-tenant scale. I wrote up the full v1-to-v2 evolution in [OPTIMIZATION_JOURNEY.md](#) if you want the implementation sketches."

Beat 10 — Close (20 seconds)

"To summarise — deterministic ranker for selection, grounded LLM for explanation, post-generation validator for safety, prompt caching and tiered models for cost. **Every number traces back. Every causal claim is hedged. Every failure mode has a specific mitigation.** I'm happy to go deeper on any of it — would you like to start with the scoring formula, the personalisation, or the failure-mode mitigations?"

Quick rehearsal notes

- **Total run-time:** ~5 minutes 0 seconds at a steady conversational pace.
- **Where to pause:** between each numbered beat (a half-second breath). After the closing question, pause **fully** — let them choose.
- **What to point at on the dashboard during the script:**
 - Beat 3 → architecture diagram in the Overview section
 - Beat 4 → EDA section ($r=0.12$ scatter, DoW chart)
 - Beat 5 → Personalisation section side-by-side
 - Beat 6 → Hallucination check section
 - Beat 7 → Failure-modes section
 - Beat 8 → Cost & scale section
- **Numbers to know cold** (write on a sticky note beside the laptop): **0.12** correlation, **91%** unattributed, **40%** DoW spread, **500%** cap, **1.3×** personalisation boost, **5/8** top-N, **₹2.50** per report, **₹15** → **₹4 lakhs/mo** at 10k with mitigations.

When they interrupt

If the panel cuts in mid-beat with a question, **answer the question fully then return to the next beat.** Don't try to push through the interruption — that signals "I'm reading a script." Treat each beat as a checkpoint; you can resume from any one of them.

What to say if you blank

"Sorry — give me a second to think." Then take an actual second. Senior candidates do this. Junior candidates fill silence with filler.

End of presentation pack. Rehearse Part B out loud three times before the panel, with a timer. The first read will run long. By the third you'll hit 5 minutes naturally.